```
block_given = false, block = nil
```

A method is called by passing its name to a receiver using reciever.method syntax. If no receiver is specified, self is assumed. The receiver checks for the method definition in its own class and then sequentially in its ancestor classes. The instance methods of included modules act as if they were in anonymous superclasses of the class that includes them. If the method is not found, Ruby invokes the method method_missing in the receiver. The default behavior defined in method_missing is to report an error and terminate the program.

When a receiver is explicitly specified in a method invocation, it may be separated from the method name using either a period (.) or, much, much, much more rarely, two colons (::). The only difference between these two forms occurs if the method name starts with an uppercase letter. In this case, Ruby will assume that receiver::Thing is an attempt to access a constant called Thing in the receiver *unless* the method invocation has an argument list between parentheses. Using :: to indicate a method call is soft- deprecated, because of its potential for confusion with constant access.

```
Foo.Bar()         #  method call
Foo.Bar           #  method call
Foo::Bar()        #  method call
Foo::Bar          #  constant access
```

Safe Navigation

Ruby defines a *safe navigation* operator, &., as in receiver&.method. If the receiver is nil, then the method is not called and the entire expression returns nil. If the receiver is not nil, then the expression proceeds normally. So, safe navigation is roughly equivalent to receiver.nil? ? nil : receiver.method, or, written more succinctly: receiver && receiver.method.

The safe navigation operator only works for the one method call, if you want to chain multiple methods safely, they all need the safe navigation operator.

```
post&.writer&.address&.country
```

You may sometimes see this referred to as the "lonely person" operator, because Matz said that he thought &. looks like a lonely person staring out into space.

Return Value

The return value of a method is the value of the last expression executed. The method in the following example returns the value of the if statement it contains, and that if statement returns the value of one of its branches.

```
def odd_or_even(val)
  if val.odd?
    "odd"
  else
    "even"
  end
end

odd_or_even(26) # => "even"
odd_or_even(27) # => "odd"
```